

# Day04. 09.08 Tue

교육 보충

## ▼ 디자인 패턴

- main.py 하나에 전부 작성하면 안돼나?

```
#ex. 로그인 + 에이전트 서비스 요청  
화면 응답  
|  
HTTP 요청 받기  
|  
입력값 검증  
|  
로그인 확인  
|  
권한 확인  
|  
업무 관련 규칙 처리  
|  
DB 조회  
|  
DB 저장  
|  
외부 API 호출 (LLM)  
|  
로그 기록  
|  
응답 만들기  
... .
```

- MVC 패턴

|       |                                                               |
|-------|---------------------------------------------------------------|
| Model | 데이터와 비즈니스 로직 (backend/, models/, services/, repositories/...) |
| View  | 사용자에게 보여주는 화면 (frontend/)                                     |

Controller                      요청을 받고 Model 과 View를 연결 (api/)

- 우리 폴더 구조는 MVC 에서 역할을 더 세분화 하여 만듦.
- MVC 패턴을 착안한 Layerd Architecture

```
frontend : Streamlit
      | HTTP 요청
API : FastAPI Router
      |
Schema : Pydantic
      |
Service : 로직처리 ----- AI 기능
      |
Repository : DB 접근
      |
Model : DB table
      |
PostgreSQL
```

## 설정과 환경 변수

- 환경변수 값을 파이썬으로 가져오는 방법

```
os.environ.get("환경변수명") -> 형변환 직접
load_dotenv()        -> 라이브러리 설치 필요
pydantic-settings 라이브러리 활용
```

- `SecreStr`

- 실수로 출력해도 값이 안보이게 숨겨주는 타입

#### ▼ `.env` 수정

```
# --- 실행 모드 -----  
APP_MODE=mock  
  
# --- Claude LLM -----  
ANTHROPIC_API_KEY=your-claude-api-key  
# 사용할 Claude 모델  
LLM_MODEL=claude-haiku-4-5  
# 최대 출력 토큰 수 (1 ~ 8192)  
MAX_TOKENS=400  
# 응답의 무작위성 지정 (0.0 ~ 1.0) : 높을수록 무작위함  
TEMPERATURE=0.0  
  
# --- 안전 장치 -----  
# 사용자 1인당 하루 최대 호출 횟수  
DAILY_CALL_LIMIT=200  
# 사용자 질문 최대 글자 수  
MAX_INPUT_CHARS=2000  
  
# --- 데이터베이스 -----  
# 초반에 사용할 SQLite (추후 변경 예정)  
DATABASE_URL=sqlite:///./app.db  
  
# --- 어플리케이션 설정 -----  
# 상세 디버그 정보 출력 여부 설정  
DEBUG=false  
  
# --- Slack등 외부 서비스로 실제 전송 허용 여부 -----  
ALLOW_EXTERNAL_SEND=false
```

#### ▼ 프로젝트 환경변수 설정 코드 적용

```
backend/app/core/config.py
```

```

from pydantic_settings import BaseSettings, SettingsConfigDict
from pydantic import Field, SecretStr
from functools import lru_cache

class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_file=".env",
        env_file_encoding="utf-8",
        extra="ignore",
        case_sensitive=False # APP_MODE 든 app_mode든 같은것으로 보게 설정
    )

    app_mode: str = Field(default="mock", pattern=r"^(mock|live)$")
    anthropic_api_key : SecretStr | None = None
    llm_model: str = "claude-haiku-4-5"
    max_token: int = Field(default=400, ge=1, le=8192)
    temperature: float = Field(default=0.0, ge=0.0, le=1.0)
    daily_call_limit: int = Field(default=200, ge=1)
    max_input_chars: int = Field(default=200, ge=1)
    database_url: str = "sqlite:///./app.db"
    debug: bool = False
    allow_external_send: bool = False

    # live 모드인지 확인 -> settings.is_live => True/Fase
    @property
    def is_live(self) -> bool:
        return self.app_mode == "live"

    @lru_cache
    def get_settings() -> Settings:
        return Settings()

```

## ▼ 환경변수 초기화 코드

```
import os

# 한개 삭제 : 환경변수가 있으면 삭제, 없어도 에러 X
os.environ.pop("APP_MODE", None)

# 여러개 삭제 : 지우고 싶은 환경변수명 리스트
keys = ["APP_MODE", "ANTHROPIC_API_KEY"]
for key in keys:
    os.environ.pop(key, None)
```

## 실습



실습 - 환경변수 값 추가

`top_k` : 정수, 기본값 3, 1 이상 20이하

`upstage_api_key` : print에서 출력안되게, 값이 없어도 되게, 기본값 None

## 예외 계층 설계

- 예외를 체계적으로 설계, API 내부에서 발생하는 에러 메시지를 데이터로 응답해주는 구조로 설계하자.
- 예외 메시지에 포함될 내용
  - 무엇이 잘못 됐는가
  - 무엇이어야 하는지
  - 실제로 받은 값

## 예외 계층 - 상속

Exception

AgentError - 우리 앱의 모든 예외 root

NotFound 404

PermissionDenied 403

ValidationFailed 422

|                      |     |
|----------------------|-----|
| GuardTripped         | 400 |
| ExternalServiceError | 502 |

#### ▼ 구현 코드

```
class AgentError(Exception):
    status_code = 400
    code = "agent_error"

    def __init__(self, message: str, *, detail: str | None = None):
        super().__init__(message) # print(e) -> 메시지 나오
        self.message = message
        self.detail = detail

# 요청한 자원이 없다
class NotFound(AgentError):
    status_code = 404
    code = "not_found"

# 자원은 있으나, 이 사용자가 접근할 수 없다.
class PermissionDenied(AgentError):
    status_code = 403
    code = "permission_denied"

# 입력값이 규칙에 맞지 않는다
class ValidationFailed(AgentError):
    status_code = 422
    code = "validation_failed"

# 외부 서비스 호출 실패
class ExternalServiceError(AgentError):
    status_code = 502
    code = "external_service_error"

e = NotFound("문서를 찾을 수 없습니다.", detail="doc_id에 id가 없음")
```

```

print("메세지 : ", e)
print("상태코드: ", e.status_code)
print("코드: ", e.code)
print("상세 : ", e.detail)

# 사용 예시
def handle(exc):
    if isinstance(exc, AgentError):
        # 우리가 의도한 상황.
        return {"status": exc.status_code, "code": exc.c
ode, "message": exc.message}
    # 우리가 예상하지 못한 예외 발생 -> 진짜 사고
    return {"status": 500, "code": "internal_error", "me
ssage": "서버 오류가 발생했습니다."}

for exc in [
    NotFound("문서 못찾음"),
    PermissionDenied("이 문서를 볼 권한이 없습니다."),
    ExternalServiceError("문서 변환 서비스에 연결하지 못하였습
니다."),
    KeyError("doc_id")
]:
    print(handle(exc))

```

### raise...from...

- 예외 발생 원인을 잃어버리지 말자

#### ▼ 코드

```

import json

def parse_bad(text):
    try:
        return json.loads(text) # 문자열 -> 파이썬 객체
    except json.JSONDecodeError as e:
        # 우리가 설계한 예외 클래스로 예외 발생시키기
        raise ValidationFailed(

```

```

        "데이터 형식이 올바르지 않습니다.", # 우리가 예외
메세지 추가
        detail=str(e) # 원래 터진 에러의 메세지 추가
(개발자용)
    ) from e # 원인 연결

try:
    parse_bad("{이건 json 아니야}")
except ValidationFailed as e:
    print("예외 메세지 : ", e)
    print("원인 : ", type(e.__cause__).__name__, " : ", e.
__cause__, )

```

## agent-app에 예외 처리 부착

- `backend/app/core/exceptions.py`

### ▼ 코드

```

# 예외 처리
class AgentError(Exception):
    status_code = 400
    code = "agent_error"

    def __init__(self, message: str, *, detail: str | No
ne = None):
        super().__init__(message) # print(e) -> 메세지 나
옴

        self.message = message
        self.detail = detail

# 요청한 자원이 없다
class NotFound(AgentError):
    status_code = 404
    code = "not_found"

# 자원은 있으나, 이 사용자가 접근할 수 없다.

```

```

class PermissionDenied(AgentError):
    status_code = 403
    code = "permission_denied"

# 입력값이 규칙에 맞지 않는다
class ValidationFailed(AgentError):
    status_code = 422
    code = "validation_failed"

# 입력 가드에 걸림 - 길이 초과, 인젝션 의심 등
class GuardTripped(AgentError):
    status_code = 400
    code = "guard_tripped"

# 호출 한도 넘었다
class RateLimited(AgentError):
    status_code = 429
    code = "rate_limited"

# 승인 없이 외부 실행 시도
class ApprovalRequired(AgentError):
    status_code = 409
    code = "approval_required"

# 모드 변경 불가 시
class ModeNotAvailable(AgentError):
    status_code = 409
    code = "mode_not_available"

# 외부 서비스 호출 실패
class ExternalServiceError(AgentError):
    status_code = 502
    code = "external_service_error"

```

## 로깅 logging

## 로그 레벨

| 레벨       | 언제                      | 예                   |
|----------|-------------------------|---------------------|
| DEBUG    | 개발 중에만 보고 싶은 상세         | 로직 처리 상세하게 보고싶을때    |
| INFO     | 정상적으로 일어난 중요한 일 (정보성)   | 문서 업로드 완료, 승인 처리 완료 |
| WARNING  | 지금은 실행되나 앞으로 예외가 될 수 있다 | 임계치에 가깝다. 재시도 발생    |
| ERROR    | 에러발생                    | LLM 요청 실패, 500에러    |
| CRITICAL | 서비스 전체가 위험              | 서버 다운 🚨             |

## logging

- 파이썬 표준라이브러리 logging 모듈을 제공한다.
- 로깅 기본 설정
  - **level** : 지정한 레벨 이상만 출력
  - **format** : 한 줄의 모양 설정
  - **force** : ipynb에서 셀을 여러번 실행해도 설정이 겹치지 않게 할때 유용

### ▼ 코드

```
import logging

# 로그 기본 설정
logging.basicConfig(
    level=logging.DEBUG,
    format="%(asctime)s %(levelname)-8s %(name)s: %(message)s",
    datefmt="%H:%M:%S",      # 시:분:초
    force=True
)

# 이름을 붙혀 로거를 하나 가져온다.
log = logging.getLogger("demo")

log.debug("디버그 모드의 debug 입니다")
log.info("인포레벨 로그 입니다.")
```

```
log.warning("경고경고 주의하세요.")
log.error("에러발생~~~~~")
```

```
# 로그 포맷 패턴
# %(asctime)s : 시간
# %(levelname)s : 로그레벨
# %(name)s : 로거이름
# %(message)s : 메시지
```

## 로그에 남기면 안되는 것

- API 키
- 비밀번호
- 토큰
- 주민번호
- 문서 원문 전체

## agent-app에 로깅 추가

- 설정은 앱 전체에 딱 한번만 한다
- `backend/app/core/logging.py`

### ▼ 코드

```
import logging
import sys

# 이미 설정했는지 기억해 두는 표식
_CONFIGURED = False

# 출력 모양
_FORMAT = "%(asctime)s %(levelname)-8s %(name)s: %(message)s"
_DATEFMT = "%H:%M:%S"
```

```

# 내 로그가 파묻히지 않게, 로그 줄여줄 라이브러리들
_NOISY = ("httpx", "httpcore", "urllib3", "asyncio")

def setup_logging(level: int = logging.INFO, stream=None) -> None:

    # global : 함수안에서 모듈의 전역변수 수정 가능하게 해주는
    # 명령어
    global _CONFIGURED
    if _CONFIGURED: # 설정을 이미 했으면 메서드 강제 종료
        return

    # 핸들러 : 로그를 어디로 내보낼지 설정
    handler = logging.StreamHandler(stream or sys.stdout)

    # 포매터 : 로그 한줄의 모양 설정
    handler.setFormatter(logging.Formatter(_FORMAT, date
    fmt=_DATEFMT))

    root = logging.getLogger() # 이름 없는 최상위 로거
    root.setLevel(level)
    root.handlers = [handler] # 기존 핸들러 밀어내고 하나만
    # 놓기

    for name in _NOISY:
        logging.getLogger(name).setLevel(logging.WARNING)

    _CONFIGURED = True

# 로그 가져다 사용할 수 있게 getter 만들기
# name에는 관례적으로 __name__ 을 넣는다 -> 로그에 모듈 경로가
# 찍혀 어느 파일에서 난 로그인지 바로 알 수 있다.
def get_logger(name: str) -> logging.Logger:
    setup_logging()
    return logging.getLogger(name)

```

```

# ipynb
import sys, logging

if "backend" not in sys.path:
    sys.path.insert(0, "backend")

# 앞 셀에서 루트 로거를 건드렸으면, 아래와 같이 깨끗이하고 새로
# 었는다.
logging.getLogger().handlers = []
for m in [k for k in list(sys.modules) if k.startswith
("app.core.logging")]:
    del sys.modules[m]

#-----
from app.core.logging import get_logger

logger = get_logger("app.services.document") # 임시 패키지
# 명
logger.info("문서 적재 완료: %s", "DOC-HR-001") # f" ->
# %s 권장
logger.warning("임계치에 근접합니다. 주의하세요.")

```

#### ▼ 활용 예시 코드

```

class AgentError(Exception):
    status_code, code = 400, "agent_error"
    def __init__(self, message, *, detail=None):
        super().__init__(message)
        self.message, self.detail = message, detail

class NotFound(AgentError):
    status_code, code = 404, "not_found"

class ExternalServiceError(AgentError):
    status_code, code = 502, "external_error"

def fake_parsing(doc_id):

```

```

    if doc_id == "DOC-BAD":
        raise ConnectionError("Connection refused: api.l
lm.ai:443")
    return f"{doc_id}의 본문 텍스트"

def ingest(doc_id):
    logger.info("적재 시작: %s", doc_id)
    try:
        text = fake_parsing(doc_id)
    except ConnectionError as e:
        logger.exception("문서 파싱 실패 : %s", doc_id)
        # 우리 타입의 예외로 변환해 발생 시키기
        raise ExternalServiceError("문서 변환 서비스에 연결
하지 못했습니다.", detail=str(e)) from e

    logger.info("적재 완료: %s", doc_id)
    return text

# 문제없는 상황
ingest("DOC-HR-001")
# 예외 발생 상황
try:
    ingest("DOC-BAD")
except AgentError as e:
    print("Agent Error")
    print(e.status_code)
    print(e.code)
    print(e.message)

```

## pytest

- 파이썬 테스트용 라이브러리(프레임워크)

내가 만든 프로그램이 예상대로 동작하는지 자동으로 검사해주는 도구

```
python -m pip install pytest
```

- pytest 에는 “이 테스트를 실행해라” 라고 등록하는 곳이 없습니다. ⇒ **이름 규칙이 곧 등록이다.**

- 파일 : `test_*.py`
- 함수 : 이름이 `test` 로 시작

```
def add(a, b):  
    return a + b  
  
print(add(10,20)) # -> 30 나오나?
```

```
#-----  
def test_add():  
    assert add(10, 20) == 30
```

기대한 값: 30  
실제 값: 30  
-> Pass

- `assert` : 검증 키워드

## HTTP



노트

HTTP 와 웹